



PrivMX User Groups

Patryk Kisielewski

Paweł Aniszewski

Wersja 0.0.1

15 kwietnia 2025

Spis treści

1	Wprowadzenie	5
1.1	Podstawy systemu	5
2	Koncepcja	7
2.1	Założenia grup	7
2.2	Tryby komunikacji	8
2.2.1	Tryb komunikacji pomiędzy użytkownikami grupy	8
2.2.2	Tryb komunikacji użytkownika spoza grupy z grupą	9
2.2.3	Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów	9
2.2.4	Rozszerzenie o komunikację użytkownika z wieloma grupami przy użyciu współdzielonych zasobów	10
2.3	Bazowe scenariusze komunikacji	11
2.3.1	Komunikacja pomiędzy użytkownikami w ramach wspólnych zasobów	11
2.3.2	Komunikacja pomiędzy użytkownikami za zasadzie skrzynki odbiorczej	11
2.3.3	Komunikacja zwrotna	11
2.4	Rozszerzenie o zewnętrznego użytkownika	12
2.4.1	Komunikacja wewnątrz grupy	12
2.4.2	Komunikacja z zewnątrz do grupy	13
2.4.3	Komunikacja w ramach współdzielonych zasobów	14
2.4.4	Komunikacja zwrotna	14
2.5	Rozszerzenie o publicznego użytkownika	15
2.5.1	Komunikacja wewnątrz grupy	15
2.5.2	Komunikacja z zewnątrz do grupy	15
2.5.3	Komunikacja w ramach współdzielonych zasobów	16
2.5.4	Komunikacja zwrotna	16
2.6	Rozszerzenie o zewnętrzną grupę	18
2.6.1	Komunikacja wewnątrz grupy	18
2.6.2	Komunikacja z zewnątrz do grupy	18
2.6.3	Komunikacja w ramach współdzielonych zasobów	18
2.7	Rozszerzenie o anonimowość	21
2.7.1	Rozszerzenie autora o grupę	21
2.7.2	Publiczny użytkownik	22
2.8	Rozszerzenie o publiczne komunikaty	22
3	Implementacja	23
3.1	Klucze	23
3.2	Wpisy kluczy	23
3.3	Użytkownik, zewnętrzny użytkownik i publiczny użytkownik	23
3.4	Grupa i zewnętrzna grupa	23
3.5	Zasób	24
3.6	Odpowiedzialność grup i zasobów	24
3.7	Dystrybucja kluczy	24
3.7.1	Dystrybucja klucza grupy	24
3.7.2	Dystrybucja klucza szyfrowania	24
3.8	Działania na grupach	25
3.8.1	Dodawanie użytkownika do grupy	25
3.8.2	Usuwanie użytkownika z grupy	25
3.8.3	Rozszerzenia	25
3.8.4	Zmiana klucza grupy	25

3.9	Działania na zasobach	25
3.9.1	Dodawanie i aktualizowanie zasobów	25
3.9.2	Usuwanie dostępu do zasobu	26
3.10	Dodawanie dostępu do zasobu	26
3.10.1	Zmiana klucza zasobu	26
3.11	Ponowne szyfrowanie zasobów po zmianie kluczy	26
3.12	Problemy dodawanie użytkownika do grupy bez historii	27
3.13	Implementacja trybów komunikacji	27
3.13.1	Tryb komunikacji pomiędzy użytkownikami grupy	27
3.13.2	Tryb komunikacji użytkownika spoza grupy	27
3.13.3	Tryb komunikacji przy użyciu współdzielonych zasobów	27
3.13.4	Rozszerzenie o komunikację użytkownika z wieloma grupami przy użyciu współdzielonych zasobów	27
3.14	Kontrola trybów	28
3.15	Rozróżnianie komunikatów	28
3.16	Listy kontroli dostępu, polityki i role	28
3.17	Podpisywanie danych grupy i wpisów klucza	28
3.18	Weryfikacja	28
3.19	Konstrukcja grup	28
A	Endpoint API	29
A.1	Wersja minimalna	29
A.1.1	Group API	29
A.1.2	Wewnętrzne struktury danych	31
A.2	Wersja pełna	32
A.2.1	Group API	32
A.2.2	Wewnętrzne struktury danych	35
B	Inne szczegóły implementacji	37
B.1	Identyfikacja elementów systemu	37

1 Wprowadzenie

1.1 Podstawy systemu

Komunikacja w systemie PrivMX polega na przesyłaniu szyfrowanych end-to-end komunikatów (**messages**) pomiędzy użytkownikami (**users**) przy użyciu ustrukturyzowanych zasobów (**resources**) na serwerze Bridge. Komunikaty są jednostkami danych (**data units**).

W tej komunikacji najważniejszy jest fakt, że odbywa się ona pomiędzy użytkownikami. Zatem można wyróżnić konkretnego nadawcę komunikatu (**sender**) oraz konkretnego odbiorcę lub wielu konkretnych odbiorców (**recipients**) tego komunikatu. Należy podkreślić fakt, iż dla każdego komunikatu zawsze jest jeden nadawca — autor komunikatu (**author**) — oraz odbiorcą jest jeden lub wielu konkretnych użytkowników.

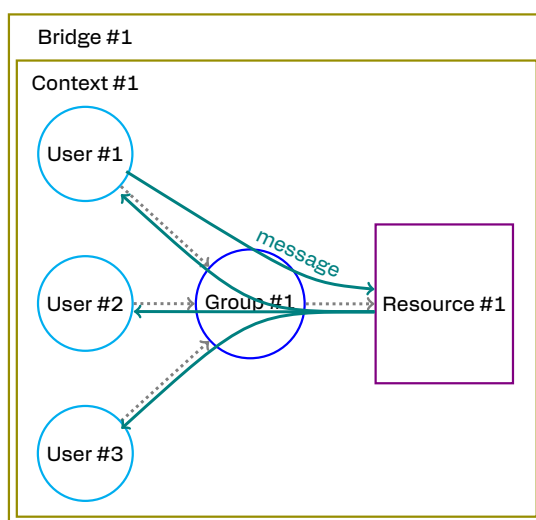
Kluczową informacją o komunikacie jest jednoznaczny i unikalny nadawca, oraz jednoznaczni i unikalni odbiorcy. Dzięki tej informacji można zweryfikować kto jest autorem komunikatu oraz dla kogo jest on zaadresowany i skierowany.

Zasoby są dodatkową abstrakcją, która porządkuje komunikaty użytkowników w większe logiczne struktury. W skład zasobów wchodzi kontenery (**containers**) oraz ich elementy (**items**). Jeden komunikat użytkownika można umieścić w jednym konkretnym zasobie.

Ważną informacją o komunikacie jest jednoznaczny i unikalny zasób, w którym umieszczony jest ten komunikat. Dzięki tej informacji można zweryfikować czy komunikat jest prawidłowo uporządkowany w logicznej strukturze.

Grupy użytkowników (**groups**) to zbiór konkretnych użytkowników mający na celu ułatwienie nadawcy adresowania komunikatów do wielu użytkowników, przesyłania i umieszczania ich w zasobach.

Ten dokument opisuje koncepcję działania grup użytkowników i jej implementację.



Rysunek 1: Basic communication.

Na Rysunek 1 przedstawiona jest bazowa komunikacja pomiędzy użytkownikami grupy, która polega na przesyłaniu komunikatów między sobą przy użyciu zasobów tej grupy.

2 Koncepcja

W tym rozdziale opisane są podstawowe założenia działania grup oraz ich rozszerzenia.

2.1 Założenia grup

Przyjęte założenia działania grup:

- Nadawcą komunikatu zawsze jest jeden konkretny użytkownik, który jest autorem tego komunikatu.
- Komunikat jest szyfrowany, podpisywany, przesyłany i umieszczany w zasobie przez autora komunikatu.
- Jeden komunikat użytkownika można umieścić w dokładnie jednym konkretnym zasobie.
- Nadawca komunikatu zawsze adresuje go do konkretnej jednej lub wielu grup.
- Odbiorcą komunikatu zawsze jest jedna lub wiele grup.
- Konkretny jeden użytkownik jest szczególnym przypadkiem jednoosobowej grupy.
- Grupa posiada swoją parę kluczy (prywatny i publiczny) kryptografii asymetrycznej.
- Jeden użytkownik może należeć do wielu grup.
- Zasoby są własnością grup.
- Jedna grupa może być właścicielem wielu zasobów.
- Jeden zasób może być własnością wielu grup.
- Kontener jest bazowym zasobem grupy.

Dodatkowe założenia użytkowe:

- Łatwe dodawanie użytkowników do grupy jest ważniejsze niż łatwe tworzenie nowych wielu grup.
- Tworzenie komunikatów, umieszczanie ich w zasobach grupy i odczytywanie komunikatów jest proste.



Rysunek 2: Base communication model.

Powyższe założenia tworzą model komunikacji, który jest przedstawiony na Rysunek 2. Użytkownik zawsze przesyła komunikat do konkretnej grupy przy użyciu zasobów.



Rysunek 3: Creating messages and resources scheme.

Rysunek 3 jest przedstawiony schemat tworzenia komunikatów i zasobów przez użytkownika. Użytkownik podający się za siebie tworzy komunikat dla grupy przy użyciu zasobu.

2.2 Tryby komunikacji

W tym podrozdziale opisane są tryby, które są konkretnymi realizacjami komunikacji zgodnej z modelem przyjętym w poprzednim podrozdziale.

Komunikacja pomiędzy użytkownikiem a grupą działa w jednym z trzech trybów:

- *Komunikacja wewnątrz grupy* - komunikacja użytkowników należących do wspólnej grupy przy użyciu jej zasobów.
- *Komunikacja z zewnątrz do grupy* - komunikacja użytkownika spoza grupy z grupą przy użyciu zasobów tej grupy.
- *Komunikacja pomiędzy grupami* - komunikacja użytkowników wielu grup przy użyciu współdzielonych zasobów pomiędzy tymi grupami.

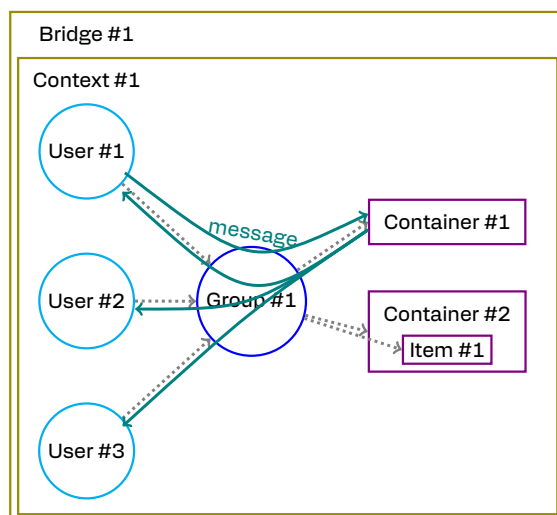
Które z trybów komunikacji są dostępne to zależy od grupy i jej zasobów. Ograniczenia są wdrożone na poziomie udostępnianych danych, użytych metod szyfrowania i podpisywania — na poziomie biblioteki — list kontroli dostępu (ACL) dla użytkowników (i grup) oraz polityk dla zasobów — serwera Bridge. Szczegóły techniczne działania trybów opisane są w podrozdziale Implementacja trybów komunikacji rozdziału Implementacja.

2.2.1 Tryb komunikacji pomiędzy użytkownikami grupy

Tryb komunikacji pomiędzy użytkownikami grupy to tryb komunikacji użytkowników należących do wspólnej grupy przy użyciu jej zasobów.

Tryb ten zakłada możliwość tworzenia wszystkich zasobów (kontenerów i ich elementów) przez użytkowników należących do grupy oraz umieszczania przez tych użytkowników komunikatów w zasobach tej grupy.

Jest to podstawowy tryb współdziałania użytkowników w ramach ich wspólnych zasobów. Te wspólne zasoby użytkowników wynikają z faktu wspólnej przynależności do tej samej grupy.



Rysunek 4: Communication between users of one group.

Na Rysunek 4 jest przedstawiony schemat komunikacji pomiędzy użytkownikami grupy. Jest to tryb typu *komunikacja wewnątrz grupy*. Komunikaty adresowane do grupy są umieszczane w zasobach tej grupy, do których dostęp mają tylko użytkownicy grupy.

W tym trybie model komunikacji (przedstawionej wcześniej) ma obostrzenia. Nadawca adresuje swoje komunikaty zawsze tylko do jednej grupy. Odbiorcą jednego komunikatu mo-

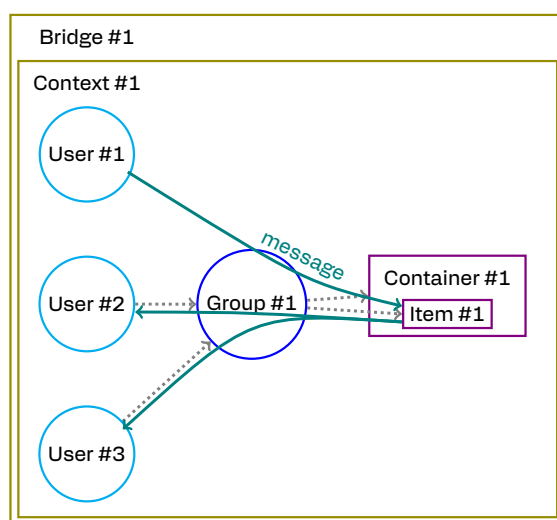
że być tylko jedna grupa. Wszystkie zasoby utworzone w ramach tej komunikacji są własnością tej konkretnej grupy.

Za dostęp użytkowników do zasobów odpowiadają użytkownicy zarządzający daną grupą. Aby nadać nowy dostęp, użytkownik może dodać nowego użytkownika do swojej grupy.

2.2.2 Tryb komunikacji użytkownika spoza grupy z grupą

Tryb komunikacji użytkownika spoza grupy z grupą to tryb komunikacji użytkownika, który nie należy do adresowanej grupy, z użytkownikami należącymi do tej grupy przy użyciu zasobów tej grupy.

Tryb ten zakłada możliwość tworzenia tylko takich zasobów, które są elementami, w konkretnych kontenerach należących do grupy i umieszczania komunikatów w tych elementach przez użytkowników nienależących do tej grupy.



Rysunek 5: Communication from outside user to group users.

Na Rysunek 5 jest przedstawiony schemat komunikacji użytkownika spoza grupy z użytkownikami tej grupy. Jest to tryb typu *komunikacja z zewnątrz do grupy*. Komunikaty adresowane do grupy są umieszczane w zasobach grupy, do których dostęp mają tylko użytkownicy tej grupy.

Tryb ten nie jest trybem współdziałania użytkowników w ramach wspólnych zasobów, ponieważ komunikaty użytkownika są umieszczane w zasobach, które są własnością adresowanej grupy oraz nie są własnością nadawcy.

W tym trybie model komunikacji ma dokładnie takie same obostrzenia jak Tryb komunikacji pomiędzy użytkownikami grupy. Nadawca adresuje swoje komunikaty zawsze tylko do jednej grupy. Odbiorcą jednego komunikatu może być tylko jedna grupa. Wszystkie zasoby utworzone w ramach tej komunikacji są własnością tej konkretnej grupy.

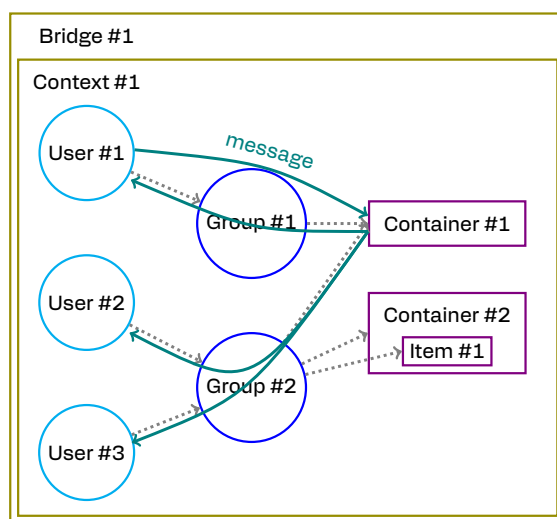
Podobnie do pierwszego trybu, za dostęp użytkowników do zasobów odpowiadają użytkownicy zarządzający daną grupą. Aby nadać nowy dostęp, użytkownik może dodać nowego użytkownika do swojej grupy.

2.2.3 Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów

Tryb komunikacji użytkowników wielu grup to tryb komunikacji pomiędzy użytkownikami należącymi do różnych grup przy użyciu współdzielonych zasobów między tymi grupami.

Tryb ten zakłada możliwość tworzenia zasobów, które są własnością więcej niż jednej grupy i umieszczania komunikatów w tych zasobach przez użytkowników należących do co najmniej jednej z tych grup.

Jest to rozszerzony tryb współdziałania użytkowników w ramach ich wspólnych zasobów. W przeciwieństwie do Trybu komunikacji pomiędzy użytkownikami grupy, wspólne zasoby użytkowników wynikają z faktu udostępniania zasobów wielu grupom na poziomie zasobów — a nie grupy.



Rysunek 6: Communication between users of different groups using shared resources.

Na Rysunek 6 jest przedstawiony schemat komunikacji użytkownika z użytkownikami wielu grup przy użyciu współdzielonych zasobów. Jest to tryb typu *komunikacja pomiędzy grupami* lub *komunikacja przy użyciu współdzielonych zasobów*. Komunikaty adresowane do wielu grup są umieszczane we współdzielonych zasobach między tymi grupami, do których dostęp mają tylko użytkownicy należący do dowolnej z tych grup.

W tym trybie model komunikacji korzysta ze wszystkich założeń opisanych wcześniej. Nadawca adresuje swoje komunikaty do wielu grup. Odbiorcą jednego komunikatu jest wiele grup. Wszystkie zasoby utworzone w ramach tej komunikacji są własnością wielu konkretnych grupy.

Działanie tego trybu jest odpowiednikiem działania Trybu komunikacji pomiędzy użytkownikami grupy dla komunikacji pomiędzy grupami.

Za dostęp użytkowników do zasobów odpowiadają użytkownicy zarządzający grupami. Aby nadać nowy dostęp, użytkownik może udostępnić zasób nowej grupie. Drugą możliwością jest dodanie nowego użytkownika do grupy w ramach standardowego zarządzania użytkownikami w swojej grupie — ta funkcja jest użyteczna do delegowania przydzielania dostępu do współdzielonego zasobu użytkownikowi innej grupy — odpowiedzialność za dostęp jest przesunięta na użytkowników zarządzających swoimi grupami.

2.2.4 Rozszerzenie o komunikację użytkownika z wieloma grupami przy użyciu współdzielonych zasobów

Tryb opisany w poprzednim punkcie rozszerza się do trybu, który zakłada możliwość tworzenia zasobów, które są własnością więcej niż jednej grupy i umieszczania komunikatów w tych zasobach przez dowolnych użytkowników.

Tryb ten nie jest trybem współdziałania użytkowników w ramach wspólnych zasobów, ponieważ komunikaty użytkownika są umieszczane w zasobach, które są własnością wielu adresowanych grup oraz nie są własnością nadawcy.

Rozszerzenie to jest odpowiednikiem Tryb komunikacji użytkownika spoza grupy z grupą dla wielu grup.

2.3 Bazowe scenariusze komunikacji

W tym podrozdziale opracowane są scenariusze komunikacji, czyli różne przypadki użycia, które są rozwiązane przy użyciu trybów komunikacji z poprzedniego podrozdziału.

Komunikacja pomiędzy użytkownikami może działać w trzech różnych bazowych scenariuszach:

- Komunikacja pomiędzy użytkownikami w ramach wspólnych zasobów.
- Komunikacja pomiędzy użytkownikami za zasadzie skrzynki odbiorczej.
- Komunikacja zwrotna.

2.3.1 Komunikacja pomiędzy użytkownikami w ramach wspólnych zasobów

W tym scenariuszu użytkownicy pracują wzajemnie ze swoimi wspólnymi zasobami takimi jak: wspólny czat użytkowników, wspólne pliki, wspólny kalendarz.

Realizacją tego scenariusza jest komunikacja wewnątrz grupy. Użytkownik może dodać tych użytkowników do grupy. Następnie komunikacja odbywa się za pomocą Tryb komunikacji pomiędzy użytkownikami grupy używając zasobów tej grupy.

2.3.2 Komunikacja pomiędzy użytkownikami za zasadzie skrzynki odbiorczej

W tym scenariuszu użytkownik wysyła zasoby do użytkowników na zasadzie skrzynki odbiorczej (w jedną stronę, bez możliwości ich późniejszego odczytania). Przykładem są wiadomości typu mail lub formularze na stronie internetowej.

Realizacją tego scenariusza jest komunikacja z zewnątrz do grupy. Komunikacja odbywa się za pomocą Tryb komunikacji użytkownika spoza grupy z grupą używając zasobów adresowanej grupy.

2.3.3 Komunikacja zwrotna

W tym scenariuszu użytkownik odpowiada innemu użytkownikowi na jego wcześniejszy komunikat. Przykładem może być wysłanie odpowiedzi na maila lub formularz.

Można wyróżnić możliwe trzy realizację tego scenariusza:

- Komunikacja w ramach nowej grupy,
- Komunikacja poprzez wysłanie komunikatów do zasobów grupy zwrotnej,
- Komunikacja w ramach współdzielonych zasobów.

Komunikacja w ramach nowej grupy

Realizacją tego scenariusza jest komunikacja wewnątrz grupy. Użytkownik, który chce wysłać komunikat zwrotny do innego użytkownika, może utworzyć nową grupę zawierającą użytkowników swojej obecnej grupy oraz nowego użytkownika. Następnie komunikacja odbywa się w ramach Tryb komunikacji pomiędzy użytkownikami grupy używając zasobów nowej grupy.

Komunikacja poprzez wysłanie komunikatów do zasobów grupy zwrotnej

Realizacją tego scenariusza jest komunikacja z zewnątrz do grupy. Pierwszy użytkownik w swoim komunikacie może załączyć tożsamość grupy zwrotnej, do której należy przesyłać odpowiedzi zwrotne, oraz tożsamość zasobu, do którego należy umieścić komunikat zwrotny. Drugi użytkownik, który chce wysłać odpowiedź, może wysłać komunikat zwrotny używając Tryb komunikacji użytkownika spoza grupy z grupą używając zasobów innej grupy.

Komunikacja w ramach współdzielonych zasobów

Realizacją tego scenariusza jest komunikacja pomiędzy grupami. Użytkownik może nadać dostęp innej grupie do konkretnych zasobów swojej grupy. Następnie komunikacja odbywa się przy użyciu Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów.

2.4 Rozszerzenie o zewnętrznego użytkownika

Organizacja to osoba lub firma, która działa w ramach własnego serwera Bridge lub własnego Kontekstu.

Opisane wyżej tryby komunikacji dotyczą wyłącznie komunikacji w obrębie jednej organizacji. W tym podrozdziale opisana jest komunikacja użytkowników należących do różnych organizacji.

Zewnętrzny użytkownik (**external user**) jest to użytkownik funkcjonujący w ramach innej organizacji. Różnicą między użytkownikiem a zewnętrznym użytkownikiem jest to, że zewnętrzny użytkownik identyfikuje się innym serwerem Bridge lub innym Kontekstem.



Rysunek 7: Communication model extended by external user.

Rozszerzenie o zewnętrznego użytkownika polega na dopuszczeniu do komunikacji zewnętrznym użytkownikom. Rozszerzony model komunikacji zaprezentowany jest na Rysunek 7. Zewnętrzny użytkownik zawsze przesyła komunikat do konkretnej grupy używając zasobów.

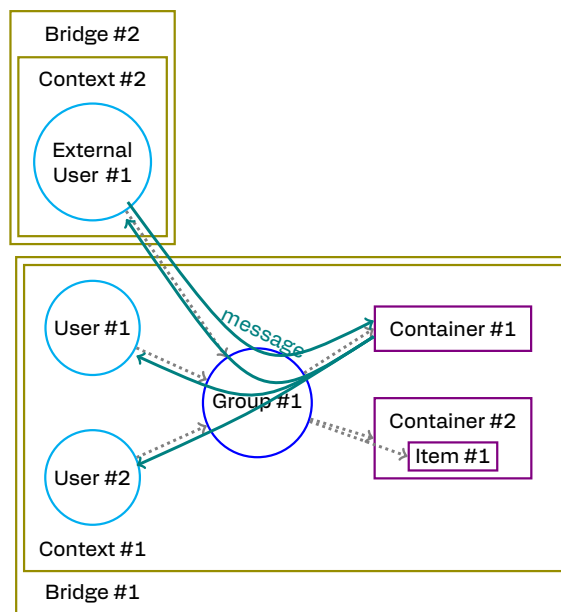
Najważniejszym zastosowaniem tego rozszerzenia jest obsługa komunikacji pomiędzy różnymi organizacjami.

2.4.1 Komunikacja wewnątrz grupy

Użytkownik może dodać do swojej grupy zewnętrznego użytkownika. Następnie komunikacja odbywa się zgodnie z Tryb komunikacji pomiędzy użytkownikami grupy używając zasobów tej grupy.

Na Rysunek 8 zaprezentowana jest komunikacja — w której zewnętrzny użytkownik należy do grupy — odbywa się zgodnie z trybem komunikacji wewnątrz grupy.

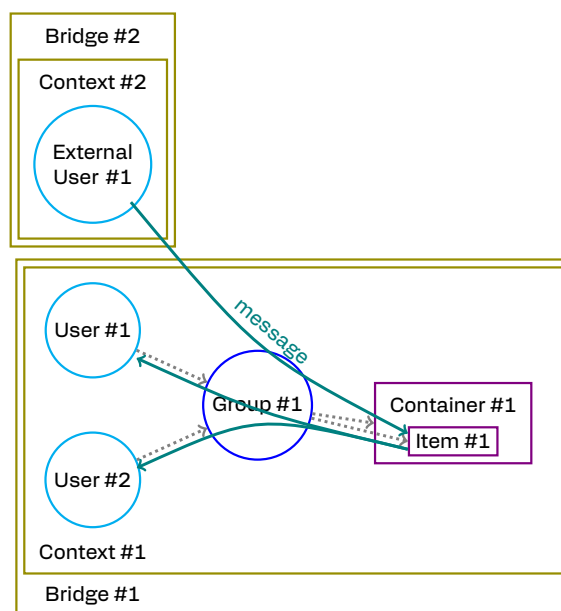
Dodanie zewnętrznego użytkownika do grupy powoduje udostępnienie temu użytkownikowi wszystkich zasobów tej grupy. Należy używać rozważnie tej funkcji. W dokumentacji należy zamieścić dobre praktyki udostępniania zasobów zewnętrznym użytkownikom.



Rysunek 8: Communication between users of the group that contains external user.

2.4.2 Komunikacja z zewnątrz do grupy

Zewnętrzny użytkownik może przestać komunikat zgodnie z Tryb komunikacji użytkownika spoza grupy z grupą używając zasobów adresowanej grupy.



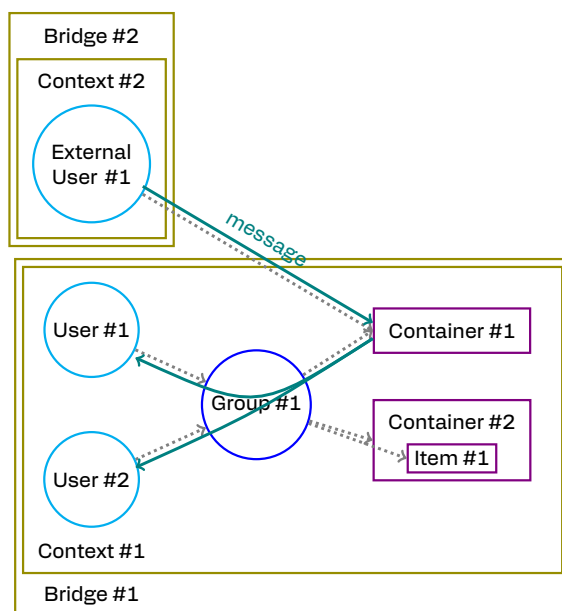
Rysunek 9: Communication from external user to group users.

Na Rysunek 9 zaprezentowana jest komunikacja — w której zewnętrzny użytkownik nie należy do grupy — odbywa się zgodnie z trybem komunikacji z zewnątrz do grupy.

W tym trybie zewnętrzny użytkownik nie uzyskuje dostępu do zasobów.

2.4.3 Komunikacja w ramach współdzielonych zasobów

Użytkownik może nadać zewnętrznemu użytkownikowi dostęp do wybranych zasobów swojej grupy. Następnie komunikacja odbywa się za pomocą Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów.



Rysunek 10: Communication between external user and users using shared resources.

Na Rysunek 10 zaprezentowana jest komunikacja — w której zasoby są współdzielone między grupą a zewnętrznym użytkownikiem — odbywa się zgodnie z trybem komunikacji pomiędzy grupami.

Ten tryb udostępniania zasobów zewnętrznemu użytkownikowi jest bezpieczniejszy, ponieważ udostępniana jest wybrana część zasobów grupy — a nie wszystkie.

2.4.4 Komunikacja zwrotna

Zewnętrzny użytkownik może dołączyć do swojego komunikatu tożsamość grupy zwrotnej. Grupa ta jest grupą znajdującą się na serwerze Bridge lub w Kontekście organizacji, w ramach której funkcjonuje zewnętrzny użytkownik. Następnie komunikacja odbywa się w Tryb komunikacji użytkownika spoza grupy z grupą.

2.5 Rozszerzenie o publicznego użytkownika

Opisane wyżej tryby komunikacji dotyczą wyłącznie komunikacji użytkowników funkcjonujących w ramach jednego lub wielu serwerów Bridge. W tym podrozdziale opisana jest komunikacja użytkownika nienależącego do żadnego serwera Bridge.

Publiczny użytkownik (**public user**) jest to użytkownik nienależący do żadnego serwera Bridge. Różnicą między użytkownikiem, zewnętrznym użytkownikiem a publicznym użytkownikiem jest to, że publiczny użytkownik nie identyfikuje się żadnym serwerem Bridge.

Działanie jest zbliżone do działania zewnętrznego użytkownika. Tożsamość serwera Bridge jest pomijana w tożsamości publicznego użytkownika.



Rysunek 11: Communication model extended by public user.

Rozszerzenie o publicznego użytkownika polega na dopuszczeniu do komunikacji publicznych użytkowników, co jest przedstawione na Rysunek 11. Publiczny użytkownik zawsze przesyła komunikat do konkretnej grupy używając zasobów.

Komunikacja publicznego użytkownika jest analogiczna do komunikacji zewnętrznego użytkownika.

Zastosowaniem publicznego użytkownika jest komunikacja użytkowników spoza jakiegokolwiek organizacji.

Przy użyciu publicznego użytkownika możliwymi zastosowaniami są nadawanie dostępu do zasobów konkretnej osobie, która nie należy do organizacji i identyfikuje się konkretnym kluczem publicznym, oraz rozwiązania podobne do udostępniania danych za pomocą hasła.

Innym zastosowaniem jest komunikacja w drugą stronę taka jak przesyłanie danych w ramach ankiet i formularzy.

2.5.1 Komunikacja wewnątrz grupy

Użytkownik może dodać do swojej grupy publicznego użytkownika. Następnie komunikacja odbywa się zgodnie z trybem Tryb komunikacji pomiędzy użytkownikami grupy używając zasobów tej grupy.

Na Rysunek 12 zaprezentowana jest komunikacja — w której publiczny użytkownik należy do grupy — odbywa się zgodnie z trybem komunikacji wewnątrz grupy.

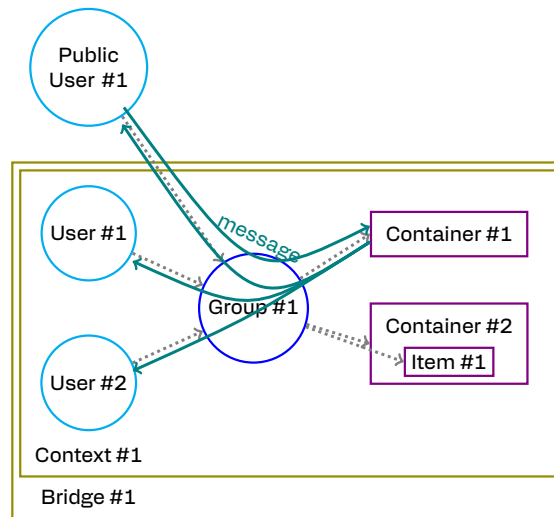
Dodanie publicznego użytkownika do grupy powoduje udostępnienie temu użytkownikowi wszystkich zasobów tej grupy. Należy używać rozważnie tej funkcji. W dokumentacji należy zamieścić dobre praktyki udostępniania zasobów publicznym użytkownikom.

2.5.2 Komunikacja z zewnątrz do grupy

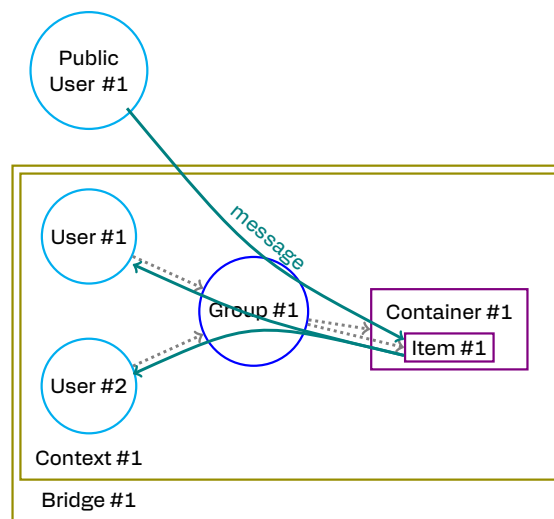
Publiczny użytkownik może wysłać komunikat używając trybu Tryb komunikacji użytkownika spoza grupy z grupą używając zasobów adresowanej grupy.

Na Rysunek 13 zaprezentowana jest komunikacja — w której publiczny użytkownik nie należy do grupy — odbywa się zgodnie z trybem komunikacji z zewnątrz do grupy.

W tym trybie publiczny użytkownik nie uzyskuje dostępu do zasobów.



Rysunek 12: Communication between users of the group that contains public user.



Rysunek 13: Communication between public user and users of the group.

2.5.3 Komunikacja w ramach współdzielonych zasobów

Użytkownik może nadać dostęp publicznemu użytkownikowi do konkretnych zasobów swojej grupy. Następnie komunikacja odbywa się za pomocą Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów.

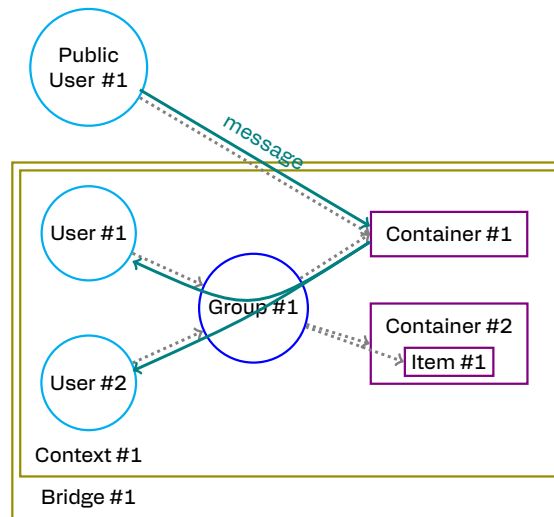
Na Rysunek 14 zaprezentowana jest komunikacja — w której zasoby są współdzielone między grupą a publicznym użytkownikiem — odbywa się zgodnie z trybem komunikacji pomiędzy grupami.

Ten tryb udostępniania zasobów publicznemu użytkownikowi jest bezpieczniejszy, ponieważ udostępniana jest wybrana część zasobów grupy — a nie wszystkie.

2.5.4 Komunikacja zwrotna

Komunikacja w ramach grupy zwrotnej nie ma zastosowania, ponieważ publiczny użytkownik nie funkcjonuje w obrębie żadnego serwera Bridge.

Istnieje opcja wskazania przez niego jakiejś innej grupy zwrotnej w celu komunikacji w Tryb komunikacji użytkownika spoza grupy z grupą.



Rysunek 14: Communication between public user and users using shared resources.

Istnieje możliwość komunikacji z publicznym użytkownikiem za pomocą dowolnego innego komunikatora, np. za pomocą maila.

Natomiast w drugą stronę, zewnętrzny użytkownik może wysłać komunikat zgodnie z Tryb komunikacji użytkownika spoza grupy z grupą używając zasobów adresowanej grupy.

2.6 Rozszerzenie o zewnętrzną grupę

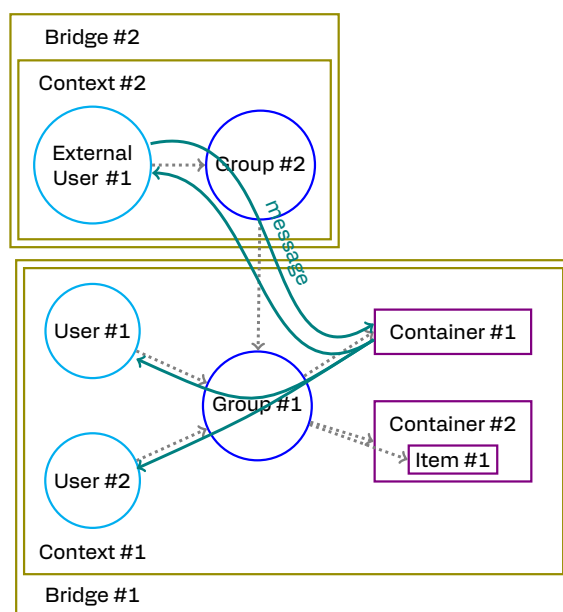
Zewnętrzna grupa (**external group**) jest to grupa funkcjonująca w ramach innej organizacji.

Rozszerzenie o zewnętrzną grupę nie zmienia modelu komunikacji i nie dopuszcza do komunikacji innych użytkowników niż użytkownik i zewnętrzny użytkownik.

Istotną różnicą między grupą a zewnętrzną grupą jest to, że w przypadku zewnętrznej grupy za przynależność użytkowników do tej grupy odpowiada zewnętrzna organizacja.

2.6.1 Komunikacja wewnątrz grupy

Użytkownik może dodać do swojej grupy zewnętrzną grupę. Następnie komunikacja odbywa się zgodnie z Tryb komunikacji pomiędzy użytkownikami grupy używając zasobów lokalnej grupy.



Rysunek 15: Communication between users of the group that contains external group.

Na Rysunek 15 zaprezentowana jest komunikacja — w której zewnętrzna grupa należy do grupy — odbywa się zgodnie z trybem komunikacji wewnątrz grupy.

Dodanie zewnętrznej grupy do grupy powoduje udostępnienie użytkownikom zewnętrznej grupy wszystkich zasobów grupy. Należy używać rozważnie tej funkcji. W dokumentacji należy zamieścić dobre praktyki udostępniania zasobów zewnętrznym grupom.

2.6.2 Komunikacja z zewnątrz do grupy

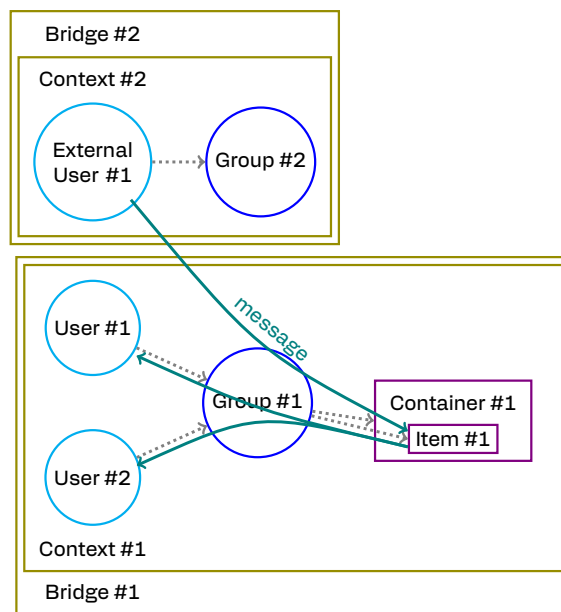
Bez zmian, ponieważ model komunikacji nie zmienił się. Grupa nie może wysyłać komunikatów. Zewnętrzny użytkownik może wysłać komunikat.

Na Rysunek 16 zaprezentowana jest komunikacja — w której zewnętrzny użytkownik należy do zewnętrznej grupy — odbywa się zgodnie z trybem komunikacji z zewnątrz do grupy.

W tym trybie zewnętrzna grupa nie uzyskuje dostępu do zasobów.

2.6.3 Komunikacja w ramach współdzielonych zasobów

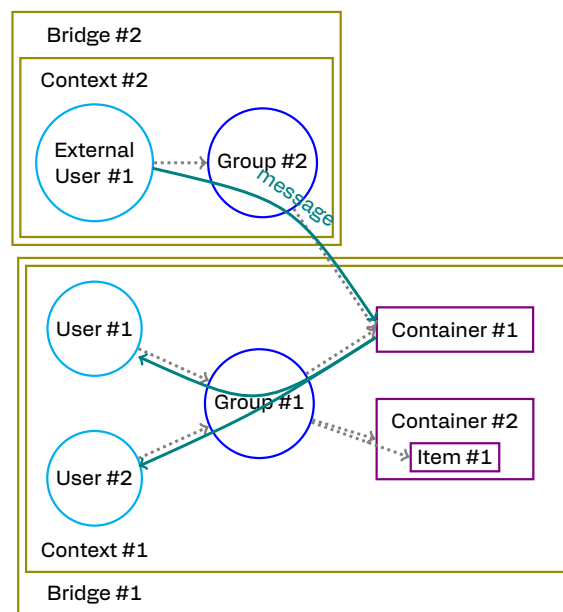
Użytkownik może nadać dostęp zewnętrznej grupie do wybranych zasobów swojej grupy. Następnie komunikacja odbywa się za pomocą Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów.



Rysunek 16: Communication between member of external group and users of the group.

Na Rysunek 17 zaprezentowana jest komunikacja — w której zasoby są współdzielone między grupą a zewnętrzną grupą — odbywa się zgodnie z trybem komunikacji pomiędzy grupami.

Ten tryb udostępniania zasobów zewnętrznej grupie jest bezpieczniejszy, ponieważ udostępniana jest wybrana część zasobów grupy — a nie wszystkie.



Rysunek 17: Communication between external group and groups using shared resources.

2.7 Rozszerzenie o anonimowość

Możliwe są dwa różne mechanizmy anonimowości użytkowników.

2.7.1 Rozszerzenie autora o grupę

Rozszerzenie autora o grupę polega na dopuszczeniu do komunikacji użytkownika podającego się za grupę.



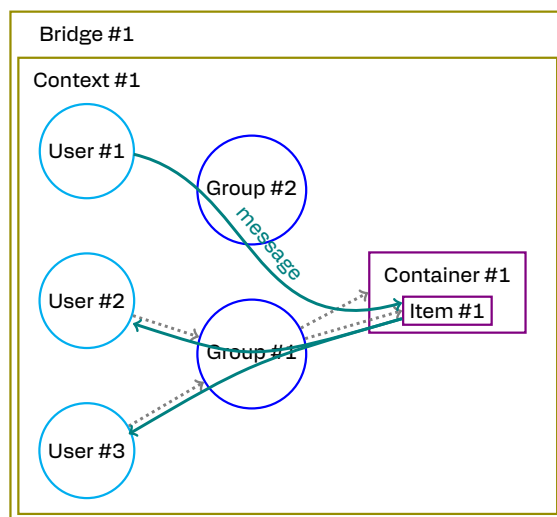
Rysunek 18: Communication model with extended author by group.

Na Rysunek 18 przedstawiony jest rozszerzony model komunikacji, w którym użytkownik jako grupa przesyła komunikat do konkretnej grupy używając zasobów.



Rysunek 19: Extended creating messages and resources scheme.

Na Rysunek 19 jest przedstawiony rozszerzony schemat tworzenia komunikatów i zasobów przez użytkownika. Użytkownik podający się za grupę tworzy komunikat dla grupy przy użyciu zasobu.



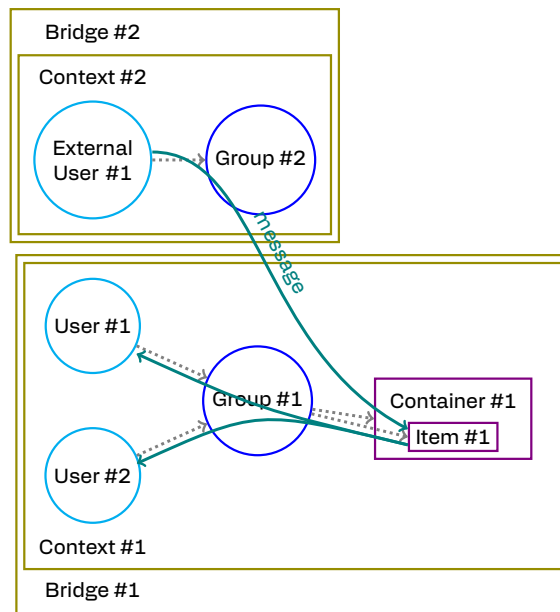
Rysunek 20: Communication between user as group and users of other group.

Na Rysunek 20 zaprezentowany jest przykład komunikacji anonimowej dla grupy.

Na Rysunek 21 zaprezentowany jest przykład komunikacji anonimowej dla zewnętrznej grupy.

Wiadomo, że autorem komunikatu jest grupa. Natomiast nie wiadomo, który konkretny użytkownik grupy jest autorem.

Zastosowaniem tego rozszerzenia mogą być grupy użytkowników ukrytych — czytających jak i piszących.



Rysunek 21: Communication between user of external group as group and users of other group.

2.7.2 Publiczny użytkownik

Publiczny użytkownik nie zawsze musi być anonimowy oraz nie zawsze można go zweryfikować. Dopóki identyfikuje się on jako ten sam publiczny użytkownik, można zweryfikować integralność jego komunikatów oraz to, że on jest ich autorem.

Publiczny użytkownik nie będzie anonimowy, kiedy osoba posługująca się konkretnym kluczem publicznym publicznego użytkownika zostanie zweryfikowana manualnie.

Serwer Bridge zezwala na nawiązywanie sesji z każdym publicznym użytkownikiem. Dodatkową anonimowość można uzyskać poprzez wysyłanie tylko jednego komunikatu (lub ich ograniczonej liczby do jednego działania na zasobach) z użyciem jednej tożsamości publicznego użytkownika.

2.8 Rozszerzenie o publiczne komunikaty

Publiczny komunikat jest to komunikat podpisany przez użytkownika lub grupę.

Użytkownik może zamieścić w zasobach swojej grupy publiczny komunikat. Następnie wszyscy użytkownicy mogą odczytywać komunikat i zweryfikować jego autentyczność.

Przykładowym zastosowaniem są publiczne: serwery plików, dystrybucja oprogramowania komputerowego, ogłoszenia instytucji rządowych lub finansowych.

Jedną z możliwości implementacji publicznych komunikatów jest udostępnienie zasobów wszystkim (wszystkim użytkownikom, wszystkim zewnętrznym użytkownikom i wszystkim publicznym użytkownikom).

Rozszerzenie o publiczne komunikaty jest rozszerzeniem Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów o zasoby udostępnione publicznie.

3 Implementacja

3.1 Klucze

Wyróżnia się trzy rodzaje kluczy, które mają różne przeznaczenie:

- Klucz użytkownika (**user key**) - para kluczy (prywatny i publiczny) kryptografii asymetrycznej.
- Klucz grupy (**group key**) - para kluczy (prywatny i publiczny) kryptografii asymetrycznej.
- Klucz szyfrowania (**encryption key**) - klucz kryptografii symetrycznej.

Klucz szyfrowania służy do szyfrowania komunikatów użytkowników oraz jest szyfrowany publicznymi kluczami grup. Klucz grupy służy do szyfrowania kluczy szyfrowania oraz jest szyfrowany publicznymi kluczami użytkowników.

3.2 Wpisy kluczy

Wpisy kluczy (**key entries**) to dodatkowe struktury danych, które reprezentują zaszyfrowane klucze umieszczone na serwerze Bridge w celu ich dystrybucji.

Wyróżnia się dwa rodzaje wpisów kluczy:

- Wpis klucza szyfrowania (**encryption key entry**) - klucz szyfrowania zaszyfrowany konkretnym kluczem publicznym grupy, w postaci (tożsamość grupy, zaszyfrowany klucz szyfrowania).
- Wpis klucza grupy (**group key entry**) - prywatny klucz grupy zaszyfrowany konkretnym kluczem publicznym użytkownika, w postaci (tożsamość użytkownika, zaszyfrowany klucz grupy).

Wpisy kluczy nie są standardowymi zasobami. Mają one konkretne ograniczenia. Wpisy te można umieszczać na serwerze tylko dla użytkowników grupy — w przypadku wpisu klucza grupy — oraz tylko dla grup uprawnionych do dostępu do zasobu — w przypadku wpisu klucza szyfrowania.

Użytkownik może pobierać z serwera tylko wpisy dotyczące jego — w przypadku wpisów klucza grupy — oraz tylko wpisy dotyczące grup, do których należy — w przypadku wpisu klucza szyfrowania.

3.3 Użytkownik, zewnętrzny użytkownik i publiczny użytkownik

- Ma klucz użytkownika.
- Uwierzytelnia się swoim kluczem.
- Uwierzytelnia się przy tworzeniu sesji. (A nie metod takich jak `inboxSend()`).
- Ma ACL.
- Wszystkie komunikaty, które tworzy i umieszcza w zasobach użytkownik, podpisuje on ten komunikat swoim kluczem prywatnym.

3.4 Grupa i zewnętrzna grupa

- Ma klucz grupy.
- Klucz publiczny jest umieszczony w danych grupy.
- Posiada wpisy kluczy grupy.

- Klucz prywatny jest umieszczony we wpisach klucza grupy.
- Posiada historię, w tym kluczy publicznych.
- Użytkownicy dodani do grupy mają rolę.
- Użytkownicy w grupie mają rolę.
- Ma polityki.

W ramach rozszerzonego modelu komunikacji, użytkownik może podpisywać komunikaty kluczem prywatnym grupy, może uwierzytelniać się kluczem grupy kluczem, w tym przypadku grupa ma też ACL.

3.5 Zasób

- Komunikaty umieszczone w zasobie są szyfrowane losowym kluczem szyfrowania.
- Posiada wpisy klucza szyfrowania.
- Klucz szyfrowania jest umieszczony we wpisach klucza szyfrowania.
- Ma polityki.

3.6 Odpowiedzialność grup i zasobów

Odpowiedzialnością grup jest zarządzanie dostępem dla użytkowników poprzez dystrybucję kluczy grupy między użytkowników grupy. Grupa pełni rolę odbiorcy komunikatów.

Natomiast odpowiedzialnością zasobów jest przechowywanie w swoich danych zaszyfrowanych komunikatów użytkowników oraz przechowywanie kluczy szyfrowania dla grup.

3.7 Dystrybucja kluczy

Dystrybucja kluczy odbywa się przy użyciu kryptografii asymetrycznej i wpisów kluczy.

3.7.1 Dystrybucja klucza grupy

Użytkownik, który zarządza grupą, podczas tworzenia lub aktualizacji grupy zamieszcza publiczny klucz grupy w publicznych danych grupy, szyfruje klucz prywatny grupy dla każdego użytkownika jego kluczem publicznym oraz tworzy dla nich wpisy klucza grupy, które umieszcza razem z danymi grupy. Wpisy klucza grupy umieszczane są razem z danymi grupy na serwerze Bridge. Tworzeniem tych wpisów zajmuje się zawsze użytkownik zarządzający grupą.

3.7.2 Dystrybucja klucza szyfrowania

Użytkownik podczas tworzenia lub aktualizacji zasobu, szyfruje klucz szyfrowania dla każdej grupy jej kluczem publicznym oraz tworzy dla nich wpisy klucza szyfrowania, które umieszcza razem z danymi zasobu. Wpisy klucza zasobu umieszczane są razem z danymi zasobu na serwerze Bridge. Tworzeniem tych wpisów zajmuje się zawsze użytkownik tworzący lub aktualizujący zasoby.

3.8 Działania na grupach

3.8.1 Dodawanie użytkownika do grupy

1. Weź wszystkie klucze prywatne grupy lub wygeneruj losowy (w zależności od wybranego trybu).
2. Zaszyfruj klucze prywatne grupy nowemu użytkownikowi.
3. Podpisz wpisy kluczy grupy.
4. Zaktualizuj wpisy kluczy grupy razem z grupą na serwerze Bridge.

Dodawanie użytkowników do grupy odbywa się zgodnie z Dystrybucją klucza grupy.

3.8.2 Usuwanie użytkownika z grupy

1. Wygeneruj losowy klucz prywatny grupy.
2. Zaszyfruj nowy klucz prywatny grupy wszystkim użytkownikom.
3. Zapisz dla każdego użytkownika (tożsamość użytkownika, zaszyfrowany klucz) razem z grupą.
4. Opcjonalnie zaktualizuj wszystkie zasoby grupy. (Patrz Ponowne szyfrowanie zasobów po zmianie kluczy.)

3.8.3 Rozszerzenia

Można w analogiczny sposób dodawać zewnętrznych użytkowników, zewnętrzne grupy i publicznych użytkowników.

3.8.4 Zmiana klucza grupy

Klucz grupy powinien zostać zmieniony w przypadku:

- Klucz prywatny dowolnego użytkownika został zmieniony.
- Klucz prywatny dowolnego użytkownika został skompromitowany.
- Klucz prywatny grupy został skompromitowany.

Rozszerzenie:

- Klucz prywatny dowolnego zewnętrznego użytkownika został zmieniony.
- Klucz prywatny dowolnego zewnętrznego użytkownika został skompromitowany.
- Klucz prywatny dowolnej zewnętrznej grupy został zmieniony.
- Klucz prywatny dowolnej zewnętrznej grupy został skompromitowany.
- Klucz prywatny dowolnego publicznego użytkownika został skompromitowany.

3.9 Działania na zasobach

3.9.1 Dodawanie i aktualizowanie zasobów

Aby dodać lub zaktualizować zasobów:

1. Wygeneruj losowy klucz szyfrowania.
2. Zaszyfruj komunikat kluczem szyfrowania.

3. Zaszzyfruj klucz szyfrowania każdej grupie, do której adresowany jest komunikat.
4. Podpisz zaszyfrowany komunikat oraz wpisy klucza szyfrowania.
5. Zaktualizuj wpisy szyfrowania razem z zasobem na serwerze Bridge.

Dodawanie i aktualizowanie zasobów odbywa się zgodnie z Dystrybucją klucza szyfrowania.

3.9.2 Usuwanie dostępu do zasobu

Ma zastosowanie w przypadku założenia, że zasoby mogą być własnością wielu grup.

Aby usunąć dostęp dowolnej grupie, należy:

1. Odebrać dostęp do zasobu tej grupie.
2. Opcjonalnie wygenerować nowy klucz i ponownie zaszyfrować komunikat. (Patrz Ponowne szyfrowanie zasobów po zmianie kluczy.)

3.10 Dodawanie dostępu do zasobu

Ma zastosowanie w przypadku założenia, że zasoby mogą być własnością wielu grup.

Aby dodać dostęp dowolnej grupie, należy:

1. Dodać dostęp do zasobu tej grupie.
2. Zaszzyfruj klucz szyfrowania nowej grupie, do której przyznawany jest dostęp.
3. Podpisz nowy wpis klucza szyfrowania.
4. Zaktualizuj wpis szyfrowania w zasobie na serwerze Bridge.

Dodawanie dostępu do zasobów odbywa się zgodnie z Dystrybucją klucza szyfrowania.

3.10.1 Zmiana klucza zasobu

Klucz szyfrowania zasobu powinien zostać zmieniony w przypadku:

- Klucz prywatny dowolnej grupy został zmieniony. (Patrz Problemy dodawanie użytkownika do grupy bez historii.)
- Klucz prywatny dowolnej grupy został skompromitowany.
- Został odebrany dostęp dowolnej grupie.

3.11 Ponowne szyfrowanie zasobów po zmianie kluczy

Jeżeli użytkownik miał dostęp do danych, to można założyć, że zdążył sobie zrobić wielokrotną kopię tych danych.

W celu ochrony danych: Pierwszą linią obrony jest serwer Bridge, który nie zwróci użytkownikowi zasobów. Kolejnym krokiem jest zmiana kluczy grupy i/lub ponowne szyfrowanie zasobów nowymi kluczami szyfrowania.

Ponowne szyfrowanie zasobu jest skomplikowane i kosztowne, bo musi to zrobić osoba, która ma do tych zasobów dostęp oraz jest w posiadaniu klucza deszyfrującego.

Ale jest możliwe napisanie aktualizacji grup i zasobów jako dodatkowa funkcja zgodnie z warunkami opisanymi wyżej.

Można dodać mechanizm wsparcia przez serwer wyszukiwania zasobów do ponownego szyfrowania.

3.12 Problemy dodawanie użytkownika do grupy bez historii

Ten tryb działania grupy może powodować nieoczywiste problemy, niespójności lub efekty uboczne. Należy wykonać dodatkową analizę tego problemu oraz opracować rozwiązanie. Przykładowym rozwiązaniem może być utrzymywanie wersji kluczy, a wymiana klucza polegałaby na zrobieniu nowej wersji klucza i ponownym szyfrowaniu zasobów.

Zdaje się też, że dodanie założenia, że wszyscy mają taki sam dostęp do kluczy szyfrowania, uprasza działanie.

3.13 Implementacja trybów komunikacji

W tym podrozdziale opisane są implementacje trybów komunikacji opisanych w rozdziale Koncepcja. Opisane jest też domyślne działanie serwera Bridge.

3.13.1 Tryb komunikacji pomiędzy użytkownikami grupy

W Tryb komunikacji pomiędzy użytkownikami grupy tworzenie i modyfikowanie zasobów odbywa się zgodnie z Dodawanie i aktualizowanie zasobów. Serwer zezwala na tworzenie, aktualizację i czytanie zasobów tylko użytkownikom należącym do grupy i przyjmuje wpisy szyfrowania tylko dotyczące tej grupy.

W celu dodatkowej weryfikacji zasobu można załączyć w nim podpis wykonany przy użyciu klucza prywatnego grupy. Podpis ten może być weryfikowany przez serwer Bridge lub użytkowników czytających zasób w celu potwierdzenia, że autor komunikatu należał do grupy.

3.13.2 Tryb komunikacji użytkownika spoza grupy

W Tryb komunikacji użytkownika spoza grupy z grupą tworzenie i modyfikowanie zasobów odbywa się zgodnie z Dodawanie i aktualizowanie zasobów. Serwer zezwala na tworzenie wpisów wszystkim użytkownikom w konkretnych kontenerach grupy. Serwer zezwala na aktualizację i czytanie zasobów tylko użytkownikom należącym do grupy. Serwer zawsze przyjmuje wpisy szyfrowania tylko dotyczące tej grupy.

3.13.3 Tryb komunikacji przy użyciu współdzielonych zasobów

W Tryb komunikacji użytkowników wielu grup przy użyciu współdzielonych zasobów tworzenie i modyfikowanie zasobów odbywa się zgodnie z Dodawanie i aktualizowanie zasobów. Serwer zezwala na tworzenie, aktualizację i czytanie zasobów tylko użytkownikom należącym do co najmniej jednej z grup, którym udostępniony jest kontener i przyjmuje wpisy szyfrowania tylko dotyczące tych grup.

3.13.4 Rozszerzenie o komunikację użytkownika z wieloma grupami przy użyciu współdzielonych zasobów

W Rozszerzenie o komunikację użytkownika z wieloma grupami przy użyciu współdzielonych zasobów tworzenie i modyfikowanie zasobów odbywa się zgodnie z Dodawanie i aktualizowanie zasobów. Serwer zezwala na tworzenie wpisów wszystkim użytkownikom i przyjmuje wpisy szyfrowania tylko dotyczące tych grup, które są właścicielem kontenera, oraz opcjonalnie grupy użytkownika autora. Serwer zezwala na aktualizację i czytanie zasobów tylko użytkownikom należącym do co najmniej jednej z grup, którym udostępniony jest zasób, i przyjmuje wpisy szyfrowania tylko dotyczące tych grup.

3.14 Kontrola trybów

Które z trybów komunikacji są dostępne to zależy od grupy i jej zasobów. Ograniczenia są wdrożone na poziomie udostępnianych danych, użytych metod szyfrowania i podpisywania — na poziomie biblioteki — list kontroli dostępu (ACL) dla użytkowników (i grup) oraz polityk dla zasobów — serwera Bridge. Szczegóły techniczne działania trybów opisane są w podrozdziale Implementacja trybów komunikacji rozdziału Implementacja.

3.15 Rozróżnianie komunikatów

Rozpoznanie w jakim trybie dany komunikat został umieszczony w zasobie można przeprowadzić w następujące sposoby:

- Jeżeli komunikat jest podpisany kluczem grupy, to znaczy, że jest on częścią komunikacji wewnątrz grupy, a autor należy do grupy.
- Jeżeli autor nie należy do grupy, to znaczy, że komunikat jest częścią komunikacji z zewnątrz.

Autorów wiadomości należy weryfikować.

3.16 Listy kontroli dostępu, polityki i role

3.17 Podpisywanie danych grupy i wpisów klucza

Podpis wpisu klucza obejmuje tożsamość autora wpisu, tożsamość przeznaczenia (użytkownika — dla wpisu klucza grupy — lub grupy — dla wpisu klucza szyfrowania), oraz miejsce zamieszczenia wpisu (grupa — dla wpisu klucza grupy — lub zasób — dla wpisu klucza szyfrowania). Podpis danych grupy obejmuje tożsamość autora wpisu i tożsamość grupy.

Dodatkowo każdy podpis obejmuje znacznik czasu — do walidacji daty stworzenia wpisu — oraz UUID — w celu detekcji duplikatów zawartości wpisu.

3.18 Weryfikacja

Każdy komunikat (lub ich większą całość) należy zawsze weryfikować podpisy, czy są podpisane przez użytkownika, weryfikować go w PKI.

3.19 Konstrukcja grup

Grupa zawiera listę składającą się z następujących możliwych elementów:

- Użytkownik.
- Zewnętrzny użytkownik.
- Publiczny użytkownik.
- Zewnętrzna grupa.

Grupa nie może zawierać się w innej grupie. Wyjątkiem jest zewnętrzna grupa. Ograniczenie jest w celu uniknięcia cyklicznych grup.

Użytkownik, zewnętrzny użytkownik, publiczny użytkownik i zewnętrzna grupa to typy specjalne grupy, przy których taka grupa zawiera tylko jeden element i jest on danego typu, a jej klucz publiczny jest identyczny jak klucz publiczny elementu.

Taka grupa specjalna służy do uproszczenia komunikacji odbieranych komunikatów. Komunikaty skierowane do konkretnego użytkownika są kierowane do tych grup.

A Endpoint API

Ten dodatek zawiera szkice API biblioteki.

A.1 Wersja minimalna

Wersja minimalna to taki szkic Group API, który można zamodelować w miarę prosto przy użyciu obecnego Endpoint API.

A.1.1 Group API

```
struct UserIdentity {
    std::string contextId;
    std::string userId;
    std::string userPubKey;
};

struct GroupIdentity {
    std::string contextId;
    std::string groupId;
    std::string groupPubKey;
};

struct ContainerIdentity {
    std::string contextId;
    std::string resourceId;
};

struct ItemIdentity {
    std::string contextId;
    std::string containerResourceId;
    std::string resourceId;
};

using ResourceIdentity = std::variant<ContainerIdentity, ItemIdentity>;

enum UserRole { USER, MANAGER };

template<class T, class R>
struct With {
    T item;
    R role;
};

using UserWithRole = With<UserIdentity, UserRole>;

struct Group {
    std::string groupId;
    std::string groupPubKey;
    std::vector<UserWithRole> users;
```

```

    core::Buffer privateMeta;
    core::Buffer publicMeta;
    int64_t version;
    UserIdentity owner;
};

struct GroupPolicy {
    // ...
};

class GroupApi {

    GroupIdentity getMyGroup();

    std::string createGroup(
        const std::string& contextId,
        const std::vector<UserWithRole>& users,
        const core::Buffer& publicMeta,
        const core::Buffer& privateMeta,
        const std::optional<GroupPolicy>& policy = std::nullopt);

    void updateGroup(
        const std::string& groupId,
        const std::optional<const core::Buffer>& publicMeta,
        const std::optional<const core::Buffer>& privateMeta,
        const int64_t version,
        const bool force,
        const std::optional<core::ContainerPolicy>& policies = std::nullopt);

    void updateUsersInGroup(
        const std::string& groupId,
        const std::vector<UserWithRole>& usersToAddOrUpdate,
        const std::vector<UserIdentity>& usersToRemove);

    void updateMyIdentity(
        const std::string& groupId,
        const AnyUserIdentity& userIdentity);

    void generateNewGroupKey(
        const std::string& groupId,
        const std::vector<UserWithRole>& users);

    core::PagingList<Group> listGroups(
        const std::string& contextId,
        const core::PagingQuery& pagingQuery);

    Group getGroup(
        const std::string& groupId);

    void deleteGroup(
        const std::string& groupId);

```

```

};

class ThreadApi {
    /**
     * @param groups Jedna grupa w ramach trybu komunikacji
     * w grupie lub do grupy.
     */
    std::string createThread(
        const std::string& contextId,
        const GroupIdentity& group,
        const core::Buffer& publicMeta,
        const core::Buffer& privateMeta,
        const std::optional<core::ContainerPolicy>& policies = std::nullopt);

    /**
     * @param groups Dodatkowe grupy w celu udostępnienia
     * wiadomości w ramach trybu współdzielonych zasobów.
     */
    std::string sendMessage(
        const std::string& threadId,
        const core::Buffer& publicMeta,
        const core::Buffer& privateMeta,
        const core::Buffer& data);

    // ...
};

```

A.1.2 Wewnętrzne struktury danych

```

struct GroupKeyEntryStamp {
    // from
    UserIdentity author;
    // to
    UserIdentity user;
    // place
    GroupIdentity group;
    // when
    int64_t timestamp;
    // client ID
    std::string keyId;
    // duplication
    std::string randomId;
};

struct GroupKeyEntry {
    // server id
    std::string groupKeyEntryId;
    // from
    UserIdentity author;
    // stamp
    std::string stamp;
};

```

```

    // stamp signature
    std::string stampSignature;
    // client id
    std::string keyId;
    // key
    std::string key;
};

struct EncryptionKeyEntryStamp {
    // from
    UserIdentity author;
    // to
    GroupIdentity group;
    // place
    ResourceIdentity resource;
    // when
    int64_t timestamp;
    // client ID
    std::string keyId;
    // duplication
    std::string randomId;
};

struct EncryptionKeyEntry {
    // server id
    std::string encryptionKeyEntryId;
    // for
    GroupIdentity group;
    // stamp
    std::string stamp;
    // stamp signature
    std::string stampSignature;
    // client id
    std::string keyId;
    // key
    std::string key;
};

```

A.2 Wersja pełna

Wersja pełna to szkic Group API, który pozwala zamodelować wszystkie tryby komunikacji opisane w tym dokumencie.

A.2.1 Group API

```

struct BridgeIdentity {
    std::string bridgeUrl;
    std::string bridgeId;
    std::string bridgePubKey;
};

```



```

struct ContextIdentity {
    BridgeIdentity bridge;
    std::string contextId;
};

struct UserIdentity {
    ContextIdentity context;
    std::string userId;
    std::string userPubKey;
};

struct PublicUserIdentity {
    std::string userPubKey;
};

struct GroupIdentity {
    ContextIdentity context;
    std::string groupId;
    std::string groupPubKey;
};

struct ContainerIdentity {
    ContextIdentity context;
    std::string resourceId;
};

struct ItemIdentity {
    ContainerIdentity container;
    std::string containerResourceId;
    std::string resourceId;
};

using ResourceIdentity = std::variant<ContainerIdentity, ItemIdentity>;
using AnyUserIdentity = std::variant<UserIdentity, PublicUserIdentity>;

enum UserRole { USER, MANAGER };

enum GroupRole { READ, WRITE, PUSH };

template<class T, class R>
struct With {
    T item;
    R role;
};

using AnyUserWithRole = With<AnyUserIdentity, UserRole>;
using GroupWithRole = With<GroupIdentity, GroupRole>;

struct Group {
    std::string groupId;
    std::string groupPubKey;
};

```

```

    std::vector<AnyUserWithRole> users;
    core::Buffer privateMeta;
    core::Buffer publicMeta;
    int64_t version;
    UserIdentity owner;
};

struct GroupPolicy {
    // ...
};

class GroupApi {

    UserIdentity getUserIdentityForCurrentBridge(
        const std::string& contextId,
        const std::string& userId,
        const std::string& userPubKey);

    std::string createGroup(
        const std::string& contextId,
        const std::vector<AnyUserWithRole>& users,
        const core::Buffer& publicMeta,
        const core::Buffer& privateMeta,
        const std::optional<GroupPolicy>& policy = std::nullopt);

    void updateGroup(
        const std::string& groupId,
        const std::optional<const core::Buffer>& publicMeta,
        const std::optional<const core::Buffer>& privateMeta,
        const int64_t version,
        const bool force,
        const std::optional<core::ContainerPolicy>& policies = std::nullopt);

    void updateUsersInGroup(
        const std::string& groupId,
        const std::vector<AnyUserWithRole>& usersToAddOrUpdate,
        const std::vector<AnyUserIdentity>& usersToRemove);

    void updateMyIdentity(
        const std::string& groupId,
        const AnyUserIdentity& userIdentity);

    void generateNewGroupKey(
        const std::string& groupId,
        const std::vector<AnyUserWithRole>& users);

    core::PagingList<Group> listGroups(
        const std::string& contextId,
        const core::PagingQuery& pagingQuery);

    Group getGroup(

```

```

        const std::string& groupId);

void deleteGroup(
    const std::string& groupId);
};

class ThreadApi {
/**
 * @param groups Jedna grupa w ramach trybu komunikacji
 * w grupie lub do grupy i/lub dodatkowe grupy w celu
 * udostępnienia wiadomości w ramach trybu współdzielonych zasobów.
 */
std::string createThread(
    const std::string& contextId,
    const std::vector<GroupWithRole>& groups,
    const core::Buffer& publicMeta,
    const core::Buffer& privateMeta,
    const std::optional<core::ContainerPolicy>& policies = std::nullopt);

/**
 * @param groups Dodatkowe grupy w celu udostępnienia
 * wiadomości w ramach trybu współdzielonych zasobów.
 */
std::string sendMessage(
    const std::string& threadId,
    const core::Buffer& publicMeta,
    const core::Buffer& privateMeta,
    const core::Buffer& data,
    const std::optional<std::vector<GroupWithRole>>& groups = std::nullopt);

    // ...
};

```

A.2.2 Wewnętrzne struktury danych

```

struct GroupKeyEntryStamp {
    // from
    AnyUserIdentity author;
    // to
    AnyUserIdentity user;
    // place
    GroupIdentity group;
    // when
    int64_t timestamp;
    // client ID
    std::string keyId;
    // duplication
    std::string randomId;
};

struct GroupKeyEntry {

```

```

    // server id
    std::string groupKeyEntryId;
    // from
    AnyUserIdentity author;
    // stamp
    std::string stamp;
    // stamp signature
    std::string stampSignature;
    // client id
    std::string keyId;
    // key
    std::string key;
};

struct EncryptionKeyEntryStamp {
    // from
    AnyUserIdentity author;
    // to
    GroupIdentity group;
    // place
    ResourceIdentity resource;
    // when
    int64_t timestamp;
    // client ID
    std::string keyId;
    // duplication
    std::string randomId;
};

struct EncryptionKeyEntry {
    // server id
    std::string encryptionKeyEntryId;
    // for
    GroupIdentity group;
    // stamp
    std::string stamp;
    // stamp signature
    std::string stampSignature;
    // client id
    std::string keyId;
    // key
    std::string key;
};

```

B Inne szczegóły implementacji

B.1 Identyfikacja elementów systemu

Ważne jest, aby w stęplach i w danych zamieszczać pełne dane identyfikujące jednoznacznie elementy systemu — serwera, Kontekstu, użytkowników, grup, zasobów.

Serwer Bridge

Lista danych identyfikująca serwer Bridge:

- URL serwera Bridge (**Bridge URL**) - adres serwera Bridge zawierający protokół, nazwę hosta lub adres serwera, port lub jego brak, oraz ścieżkę dostępu zakończona ukośnikiem;
- ID serwera Bridge (**Bridge ID**) - nadany identyfikator w dowolnym formacie dopuszczającym znaki: [a-zA-Z0-9-], np. UUID, zapisany w konfiguracji serwera Bridge;
- Klucz publiczny serwera Bridge (**Bridge public key**) - klucz publiczny serwera Bridge.

Przykłady prawidłowych URL serwera Bridge:

- `http://localhost/`
- `http://localhost:3000/`
- `https://localhost:3000/`
- `http://localhost:3000/subdirectory/`
- `http://127.0.0.1/`
- `https://bridge.privmx.dev/`

Alternatywnie, zamiast URL serwera Bridge można zastosować samą nazwę domenową.

Problemy:

- URL serwera Bridge może zmieniać się w czasie, gdy administrator będzie miał taką potrzebę.
- ID serwera Bridge nie może ulegać zmianie.
- Mogą działać dwa niezależne serwery Bridge z tym samym ID. Nie ma możliwości zablokowania tego.
- Klucz publiczny serwera Bridge może zmieniać się w przypadku kompromitacji odpowiadającego mu klucza prywatnego.
- Klucz publiczny serwera Bridge może być wykorzystywany do weryfikacji jego autentyczności przez użytkownika podczas tworzenia połączenia (a dokładniej sesji).

Dane identyfikujące serwer Bridge mogą być przechowywane w PKI.

Reprezentacja tożsamości serwera Bridge:

```
BridgeIdentity {  
    bridgeUrl: string,  
    bridgeId: string,  
    bridgePubKey: string  
}
```

Kontekst

Lista danych identyfikujących jednoznacznie unikalny kontekst:

1. Bridge - konkretny serwer Bridge.
2. ID kontekstu (**Context ID**) - unikalny na serwerze Bridge identyfikator kontekstu.

Reprezentacja tożsamości kontekstu:

```
ContextIdentity {  
    bridge: BridgeIdentity,  
    contextId: string  
}
```

Użytkownik

Lista danych identyfikujących jednoznacznie unikalnego użytkownika:

1. Kontekst - unikalny kontekst.
2. ID użytkownika (**user ID**) - unikalny w konkretnym kontekście identyfikator użytkownika.
3. Klucz publiczny użytkownika (**user public key**) - klucz publiczny użytkownika.

Dane identyfikujące użytkownika mogą być przechowywane w PKI.

Reprezentacja tożsamości użytkownika:

```
UserIdentity {  
    context: ContextIdentity,  
    userId: string,  
    userPubKey: string  
}
```

Publiczny użytkownik

Lista danych identyfikujących publicznego użytkownika:

1. Klucz publiczny użytkownika (**user public key**) - klucz publiczny użytkownika.

Reprezentacja tożsamości użytkownika:

```
PublicUserIdentity {  
    userPubKey: string  
}
```

Grupa

Lista danych identyfikujących jednoznacznie unikalną grupę:

1. Kontekst - unikalny kontekst.
2. ID grupy (**group ID**) - unikalny w konkretnym kontekście identyfikator grupy.
3. Klucz publiczny grupy (**group public key**) - klucz publiczny grupy.

Dane identyfikujące grupę mogą być przechowywane w PKI.

Reprezentacja tożsamości grupy:

```
GroupIdentity {  
    context: ContextIdentity,  
    groupId: string,  
    groupPubKey: string  
}
```

Kontener

Lista danych identyfikujących jednoznacznie unikalny kontener:

1. Kontekst - unikalny kontekst.
2. ID kontenera (**container ID**) - unikalny w konkretnym kontekście identyfikator kontenera.

Reprezentacja tożsamości kontenera:

```
ContainerIdentity {  
    context: ContextIdentity,  
    containerId: string  
}
```

Element

Lista danych identyfikujących jednoznacznie unikalny element:

1. Kontener - unikalny kontener.
2. ID elementu (**item ID**) - unikalny w konkretnym kontenerze identyfikator elementu.

Reprezentacja tożsamości elementu:

```
ItemIdentity {  
    container: ContainerIdentity,  
    itemId: string  
}
```